

LEETCODE DASHBOARD ISSUE FIX REPORT

This report explains the problems that existed in the project earlier, why they were risky, and how each one was fixed in a beginner-friendly way.

1. STARTUP 'RUN()' METHOD COULD BLOCK THE WHOLE APPLICATION

WHAT WAS HAPPENING BEFORE

The Spring Boot application class implemented 'CommandLineRunner'. Inside its 'run()' method, the code kept trying to fetch the daily problem in a 'while' loop until the result was not 'null'.

That meant:

- if LeetCode was down, startup never finished
- if the network failed, startup never finished
- if some parsing error happened, startup never finished
- the application could stay stuck in boot forever

WHY THIS WAS A PROBLEM

When an application starts, it should either:

- start successfully, or
- fail clearly and quickly

It should not keep looping forever without a limit. That kind of code is dangerous because it hides the real error and makes the application look frozen.

HOW IT WAS FIXED

The 'run()' method was changed to:

- try only once during startup
- store the daily problem if the fetch succeeds

- log a warning if the fetch fails
- let the application continue starting

WHY THE NEW VERSION IS BETTER

Now the application is not held hostage by one external API call. Startup is stable, and the scheduler can retry later.

2. 'DAILYPROBLEMHOLDER.GETCURRENTPROBLEM()' COULD BE 'NULL'

WHAT WAS HAPPENING BEFORE

The method used by '/ispotdsolved/{username}' directly read:

- 'dailyProblemHolder.getCurrentProblem()'

and then immediately used:

- 'dailyPotd.getProblemId()'

If the daily problem had not been loaded yet, the holder was 'null', and the code threw a 'NullPointerException'.

WHY THIS WAS A PROBLEM

This can happen in normal situations:

- the app just started
- startup fetch failed
- scheduler has not run yet

So the endpoint was fragile even when the user did nothing wrong.

HOW IT WAS FIXED

The method now:

- checks whether the holder is 'null'
- if it is 'null', it tries to fetch the daily problem on demand
- stores the fetched problem back in the holder
- throws a controlled exception if the fetch still fails

WHY THE NEW VERSION IS BETTER

The endpoint no longer crashes with a null pointer just because the holder was empty.

3. N+1 REMOTE CALLS IN RECENT SOLVED PROBLEMS

WHAT WAS HAPPENING BEFORE

The code first fetched the list of recent submissions. Then, for every accepted submission, it made another remote GraphQL call to get full problem details by 'titleSlug'.

That pattern is called an N+1 problem:

- 1 remote call to fetch the submission list
- N more remote calls for N accepted submissions

WHY THIS WAS A PROBLEM

This causes:

- slower responses
- unnecessary API traffic
- repeated calls for the same problem if it appears multiple times
- more chances of failure because more network calls are being made

HOW IT WAS FIXED

The flow was improved in two important ways:

- only accepted submissions are processed
- problems are cached in a 'Map<String, LeetCodeProblem>' during the method execution
- the database is checked first by 'titleSlug'
- the remote GraphQL call is made only if the problem is not already in memory and not already in the database

WHY THE NEW VERSION IS BETTER

Now repeated accepted submissions for the same problem do not keep hitting LeetCode again and again. The main remote N+1 issue is solved.

4. USER STATS WERE CACHED FOREVER

WHAT WAS HAPPENING BEFORE

The method 'getUserStats(username)' looked in the database first. If a 'Student' record existed, it returned that cached value immediately and never called LeetCode again.

WHY THIS WAS A PROBLEM

That means cached data could become stale forever. A user might solve more problems on LeetCode, but your dashboard would continue showing old values.

HOW IT WAS FIXED

The 'Student' entity now has an 'updatedAt' field with '@UpdateTimestamp'.

Then the service checks whether the cached row is still fresh:

- if updated within the last 1 hour, return cached data
- otherwise call LeetCode again, refresh the row, and save it

WHY THE NEW VERSION IS BETTER

The cache now has a time limit. It improves performance without becoming permanently stale.

5. REFRESHING STALE 'STUDENT' DATA COULD CREATE DUPLICATE RECORDS

WHAT WAS HAPPENING BEFORE

When cached data became stale, the service created a brand-new 'Student' object instead of updating the existing one.

That was risky because 'username' is unique in the database. Saving a new entity with the same username could cause a duplicate key problem.

HOW IT WAS FIXED

The service now:

- creates a new 'Student' only if one does not already exist
- otherwise updates the fields on the existing entity
- saves the updated entity

WHY THE NEW VERSION IS BETTER

Refreshing stale cache now behaves like an update, not an accidental duplicate insert.

6. GRAPHQL ERROR HANDLING WAS INCOMPLETE

WHAT WAS HAPPENING BEFORE

Many methods assumed that:

- the HTTP request succeeded
- the response body existed
- the JSON had a 'data' field
- the GraphQL response had no 'errors'
- required nested fields were always present

WHY THIS WAS A PROBLEM

GraphQL often returns errors inside a successful HTTP response. So even '200 OK' can still contain failure information.

Without checking that structure, the code could:

- parse missing fields
- use wrong defaults like '0'
- throw confusing exceptions later
- hide the real cause of the failure

HOW IT WAS FIXED

A shared GraphQL execution method was added. It now:

- sends the request
- checks for empty responses
- parses JSON once
- checks the GraphQL 'errors' array
- validates that 'data' exists
- returns parsed 'data' only when the response is valid

Helper methods were also added to require important nodes and integers before using them.

WHY THE NEW VERSION IS BETTER

Now GraphQL handling is consistent across the service. Errors are caught early and described more clearly.

7. API ERROR RESPONSES WERE TOO GENERIC

WHAT WAS HAPPENING BEFORE

The global exception handler returned '400 Bad Request' for every 'RuntimeException'.

WHY THIS WAS A PROBLEM

Not every failure is a bad request. For example:

- if the user is not found, that should usually be '404'
- if LeetCode is down, that is an upstream service issue, closer to '502'
- if the server has an internal bug, that should be '500'

Returning '400' for everything makes debugging harder and gives the client the wrong meaning.

HOW IT WAS FIXED

Custom exceptions were introduced:

- 'NotFoundException'
- 'UpstreamServiceException'

Then the global exception handler was updated to map them correctly:

- '404 Not Found'
- '502 Bad Gateway'
- '500 Internal Server Error' for uncaught runtime failures
- '400 Bad Request' for input validation problems like blank usernames

WHY THE NEW VERSION IS BETTER

Now the API communicates the kind of failure more honestly and clearly.

8. EXCEPTION MESSAGES WERE TOO BROAD

WHAT WAS HAPPENING BEFORE

Some messages were vague, such as:

- "Failed to process user stats"
- "Rating not available"
- "Failed to parse raw json"

WHY THIS WAS A PROBLEM

Broad messages make it difficult for beginners and maintainers to understand:

- what operation failed
- what data was missing
- whether the problem was network, parsing, or business logic

HOW IT WAS FIXED

Messages were rewritten to be more specific, for example:

- which GraphQL operation failed
- which username was being processed
- whether the missing data was question details, ranking data, or daily problem data

WHY THE NEW VERSION IS BETTER

The project is easier to debug and easier to explain because the failures describe themselves better.

9. USERNAME ENCODING WAS HANDLED INCORRECTLY

WHAT WAS HAPPENING BEFORE

At one stage, usernames were manually altered like URL path values, for example turning spaces into '%20'.

WHY THIS WAS A PROBLEM

GraphQL variables are sent as JSON values, not URL query strings. So manual URL encoding is

the wrong layer here.

It can also create database consistency problems if:

- the stored username is one version
- the lookup username is another version

HOW IT WAS FIXED

A shared username normalization method was added. It now:

- rejects 'null'
- rejects blank usernames
- trims leading and trailing spaces
- keeps the value as a normal string for GraphQL variables

WHY THE NEW VERSION IS BETTER

The system now treats usernames consistently in both API calls and database lookups.

10. BLOCKING HTTP CALLS HAD NO TIMEOUT OR RETRY

WHAT WAS HAPPENING BEFORE

The LeetCode requests used blocking calls like `'.block()'` without any timeout or retry policy.

WHY THIS WAS A PROBLEM

That means a request could wait forever if the upstream service became slow or unreachable.

HOW IT WAS FIXED

Timeout and retry behavior were added in two places:

- the 'WebClient' configuration now has connection, response, read, and write timeouts
- the shared GraphQL execution method now applies a request timeout and retry with backoff

Retries are limited to transient failures such as:

- network problems
- upstream '5xx' responses

WHY THE NEW VERSION IS BETTER

The application no longer waits forever for LeetCode. Temporary failures get a few safe retries, but long hangs are prevented.

FINAL SUMMARY

The project is now stronger in these areas:

- startup safety
- null safety
- caching correctness
- remote call efficiency
- GraphQL validation
- API error quality
- exception clarity
- username handling
- network resilience

In short, the application became more reliable, easier to debug, and safer for real-world use.